Status: PREPARED DRAFT

Session 13 — Study Guide: A Project Has Issues

Where the session ends

Issues exists as a complete, known-pattern resource — full CRUD, access-controlled, logged, Swagger-documented — referencing Project via a plain `projectId` field. Issues can be filtered by `projectId` and `status` together, backed by a real, measured compound index. By the end of class, Session 12's own real homework is solved and its one real bug is fixed; every existing resource behaves exactly as it did before.

The real starting point

Session 12's real classroom delivery went further than planned: every real write route on both Projects and Workspaces was already guarded, logged, and Swagger-documented by the end of CORE and teamwork themselves. `GET /projects/stats` was correctly left open as the real take-home homework. A real bug was also found in the delivered code: `WorkspacesService.update()` was missing `await` on its own slug-uniqueness check. Today opens by solving the homework and fixing that bug, live, before any new material begins.

Issues — known pattern, new domain

TS

```ts
@Schema()
export class Issue {
  @Prop({ required: true })
  id: number;

  @Prop({ required: true })
  projectId: number;

  @Prop({ required: true })
  title: string;

  @Prop({ default: '' })
  description: string;

  @Prop({ default: 'todo' })
  status: string;
}
```

Same schema shape, CRUD, controller, Guard/Swagger/Logger pattern already built twice — applied a third time, fast. The one field worth pausing on is `projectId`.

The real decision: reference, not embed

`Issue.projectId` is a plain number pointing back at `Project.id` — not Mongoose's own built-in cross-collection relationship mechanism (deliberately not covered this session; see "Not yet" below). The deciding question is never "is this a relationship" (every parent-child combination here is one) — it is: does the child collection grow without bound, independently of its parent, or does it stay small and get read together with it?

```
Project -> Issues:     UNBOUNDED growth  -> REFERENCE (this session)
Issue -> Comments:      BOUNDED, read together -> EMBED (a future session)
```

Issues under a Project accumulate without limit for the project's entire lifetime — embedding them would eventually hit MongoDB's own real document-size ceiling. Comments under an Issue, once they exist, will be embedded instead — almost always read together with their own issue, and bounded for a realistic issue.

A caveat worth remembering: this reference models the relationship — it does not enforce it. MongoDB will currently accept an Issue with `projectId: 999` even if Project 999 doesn't exist. Checking that the parent actually exists is a separate application rule, and this session deliberately does not add it.

The real proof: an index, measured

CORE added a single-field index on `projectId` only AFTER measuring the real cost without one:

```
BEFORE:  winningPlan.stage: COLLSCAN       totalDocsExamined: 180   totalKeysExamined: 0
AFTER:   winningPlan.stage: FETCH (inputStage.stage: IXSCAN)
         totalDocsExamined: 60             totalKeysExamined: 60
```

Same real query, same real 60 results either way — the difference is how much work MongoDB did to find them. This is the whole lesson: **an index is chosen because of a measured query cost, not because "schemas should have indexes."** (That single-field index was later measured again, in teamwork, and removed once it became redundant — see below. By the end of the session, the schema carries only the compound index described next.)

Filtering by two fields together

TS

```ts
async findAll(query: FindIssuesQueryDto): Promise<IssueDocument[]> {
  const filter: Record<string, unknown> = {};

  if (query.projectId !== undefined) {
    filter.projectId = query.projectId;
  }

  if (query.status !== undefined) {
    filter.status = query.status;
  }

  return this.model.find(filter);
}
```

Teamwork's own real decision: does this compound query (`projectId` AND `status` together) need a compound index beyond the single-field one? Measure first, then decide.

For THAT exact query, `projectId` and `status` are both equality filters — either field order would work for the compound query alone. `projectId` goes first because of the COMPLETE real workload: `GET /issues?projectId=X` (projectId alone) is also a real, already-existing request, and a compound index's own prefix can serve it too. This is not "the always-present equality field always goes first" as a universal rule — it's a decision tied to this app's own actual query shapes.

One index can make another redundant

Once `{ projectId: 1, status: 1 }` exists, its own `projectId` prefix serves the plain `projectId`-only query at the identical cost the single-field `projectId` index gave — so that single-field index, added back in CORE, is measured again and found to be pure overhead with no remaining benefit. It is removed: `db.issues.dropIndex("projectId_1")` in `mongosh` (the schema's own code line alone is not enough — Mongoose's `autoIndex` only ever creates missing indexes, it never drops ones removed from the schema), then the line is deleted from the schema file too. Indexes are not free; an index that only duplicates another index's own prefix is a real, ongoing cost for zero remaining read benefit.

Common beginner mistakes

- **Reaching for Mongoose's own built-in cross-collection relationship mechanism.** This app has never used Mongo's own `_id` as a public identifier; `projectId` is a plain number, resolved the same way every other lookup in this app already works.
- **Treating "reference vs. embed" as a fixed property of an entity.** It's a property of a specific relationship — Issue is referenced FROM Project, but will later embed Comments ITSELF.
- **Reading only `winningPlan.stage` and stopping there.** `totalDocsExamined` / `totalKeysExamined` are what actually prove an index helped — the stage name alone isn't the full proof.

- **Reversing a compound index's own field order, or defending it with a universal rule.** The real reason `projectId` comes first is the complete real query workload (it's also queried alone), not "the field that's never used by itself always goes first" — and `status` CAN be queried by itself (`GET /issues?status=X` is genuinely supported), it just isn't the workload this index targets.
- **Keeping an index after it becomes redundant.** More indexes are not automatically better — an unmeasured index kept "just in case" is exactly the habit this session's own lesson targets.
- **Adding an index for every new filter, reflexively.** The homework's own `title` search is the deliberate counter-example — the current `{ projectId, status }` index doesn't index `title` at all, so a title-only search gets no help from it, though its `projectId` prefix still helps when a request also supplies `projectId`.

Not yet

These exist in real database/API design, but are deliberately **not** covered this session:

- Mongoose's own built-in ObjectId-based cross-collection relationship mechanism
- Aggregation pipelines, pagination
- The atomic, derived `<ProjectKey>-<n>` issue-key scheme (flagged in the project's own specification as belonging to the later PostgreSQL stage)
- Issue fields beyond this session's own minimal shape (assignee, reporter, priority, story points)
- Comments as a real, built feature
- Custom decorators, authentication, PostgreSQL/an ORM/Next.js

Glossary

- **Reference** — a child document stores a plain field pointing back at its parent; separate collections, queried independently.
- **Embed** — a child's own data lives directly inside its parent document, typically as an array.
- **`explain("executionStats")`** — MongoDB's own real query-plan and execution-statistics report.
- **`COLLSCAN`** — the query scanned every document in the collection.
- **`IXSCAN`** — the query used an index to find candidates before fetching them.
- **Compound index** — one index covering more than one field, usable as a prefix.

Review questions

1. Why does `Issue.projectId` reference Project instead of embedding Issues inside it?
2. What three fields in `explain()` output actually prove an index helped?
3. Why was the baseline measurement taken BEFORE the index was added, not after?
4. Why does `{ projectId: 1, status: 1 }` use that field order, and not the reverse — given that both fields are equality filters in this exact query?
5. Why doesn't the homework's own `title` search get its own new index?
6. Why did the original single-field `projectId` index get removed after the compound index was added?

Answers

1. A project's own issues grow without bound for its entire lifetime; embedding them would risk MongoDB's own real document-size ceiling. A reference has no such limit.
2. `winningPlan.stage` (and `inputStage.stage` when it's `FETCH`), `totalDocsExamined`, and `totalKeysExamined` — not raw timing, which is unreliable on a small local collection.
3. Because Mongoose's own `autoIndex` creates a declared index automatically on the next connect — measuring after it exists would mean the "before" state was never actually observed.
4. For this one query, either order would work — both fields are equality filters. `projectId` comes first because of the COMPLETE real workload: `GET /issues?projectId=X` (projectId alone) is also a real, already-existing request, and this field order lets the compound index's own prefix serve that query too. It is not a universal "always-present field first" rule.
5. Because the current `{ projectId, status }` index does not index `title` at all — a title-only search gets no help from it, though its `projectId` prefix still helps narrow the candidate set when a request also supplies `projectId`.
6. Because, once the compound index existed, its own `projectId` prefix was measured to serve the plain `projectId`-only query at the identical cost the single-field index gave — keeping both would have meant paying for an index that did nothing the other one couldn't already do.

What you should be able to explain now

- Why Issues' own CRUD/Guard/Swagger/Logger needed no new explanation — a known pattern, applied to a new resource.
- Why "reference vs. embed" is decided by growth shape, not by whether something "is a relationship."
- Why an index decision must be measured, both before and after, not just declared.
- Why a compound index's own field order is a real, defensible decision tied to the complete query workload, not a memorized universal rule.
- Why an index that duplicates another index's own usable prefix should be measured and removed, not kept "just in case."